

Pokémon Swim Championship

Momento II — Diseño y Análisis

Estudiante UdeA

Mayo 2026

1. Descripción detallada de vistas e interacciones por nivel

1.1 Nivel 1 — Piscina Olímpica de Cerulean City

Vista lateral con scroll horizontal ilimitado hacia la derecha. La pantalla muestra dos carriles de natación superpuestos verticalmente: el carril inferior corresponde al jugador (Totodile) y el superior al agente rival (Psyduck). El fondo de la piscina, incluyendo las divisiones de carril, las tribunas y el cielo, se desplaza de derecha a izquierda a medida que los personajes avanzan, simulando el movimiento hacia la meta.

El jugador mantiene la velocidad de Totodile alternando las teclas A y D para simular las brazadas. Cada pulsación genera un impulso que incrementa la velocidad brevemente; si el jugador deja de presionar, la velocidad decae de forma exponencial por la resistencia del agua. La stamina del personaje baja con el uso intensivo y sube gradualmente con el reposo.

Los Magikarps aparecen en posiciones aleatorias del carril del jugador y siguen una trayectoria parabólica desde el borde de la piscina hasta caer al agua. Si Totodile colisiona con un Magikarp activo, se reduce su velocidad temporalmente. El jugador puede activar la habilidad Mordedor con la tecla E para destruir obstáculos en su radio inmediato, pero esto provoca una frenada de 2 segundos por el esfuerzo.

Al inicio de cada carrera, la habilidad Hiperactivo entra en efecto automáticamente durante 3 segundos: Totodile recibe un multiplicador de velocidad inicial que después de esos 3 segundos el efecto termina y el jugador debe mantener la velocidad por su cuenta. La carrera termina cuando uno de los dos personajes llega a la meta. Al finalizar, el agente registra el tiempo del jugador para ajustar su comportamiento en la siguiente carrera.

1.2 Físicas del Nivel 1

Física	Ecuación	Elemento asociado
Resistencia hidrodinámica	$v(t) = v_0 \cdot e^{(-k \cdot t)}$	Decaimiento de velocidad de Totodile al no bracear
Movimiento parabólico	$y = y_0 + v_y \cdot t - \frac{1}{2} \cdot g \cdot t^2$	Trayectoria del Magikarp al saltar desde el borde

1.3 Nivel 2 — Río Safari, Zona Safari

Vista cenital (desde arriba) con scroll vertical descendente. El río ocupa el centro de la pantalla y avanza de arriba hacia abajo; las orillas con vegetación son estáticas en los márgenes laterales. Totodile se controla con las teclas W, A, S, D en las cuatro direcciones. El objetivo es completar el recorrido del río antes de que el timer descendente llegue a cero.

El río aplica una corriente oscilatoria lateral sobre Totodile en todo momento: la fuerza horizontal varía de forma sinusoidal, empujando al personaje alternativamente hacia la izquierda y la derecha. El jugador debe contrarrestar esta fuerza activamente para mantener la trayectoria. Adicionalmente, existe una corriente de avance constante que empuja a Totodile hacia abajo (hacia la meta), esto ayuda a avanzar más rápido, aunque también deja menos tiempo para reaccionar cuando aparece un obstáculo.

En posiciones fijas del nivel aparecen remolinos. Cuando Totodile entra en el radio de influencia de un remolino, se le aplica una fuerza centrípeta que lo arrastra en espiral hacia el centro. El jugador debe usar las teclas para contrarrestar esta atracción y escapar antes de ser arrastrado completamente.

El nivel termina cuando Totodile llega a la meta o cuando el timer llega a cero. En ambos casos se muestra la pantalla de resultado y se transita al menú principal o a una nueva partida.

1.4 Físicas del Nivel 2

Física	Ecuación	Elemento asociado
Corriente oscilatoria	$F_x = A \cdot \sin(\omega \cdot t) + C$ (constante de avance)	Corriente lateral del río sobre Totodile
Atracción espiral	$F = k / r^2$ hacia el centro del remolino	Remolinos que atraen al personaje en espiral

2. Cambios y precisiones respecto al Momento I

La dinámica general del juego se mantiene respecto a lo descrito en el Momento I. Se realizaron algunos ajustes respecto a la propuesta inicial para definir mejor varias mecánicas del juego :

- El control de brazadas en el Nivel 1 pasa de ser genérico a usar específicamente las teclas A y D alternadas, lo que hace más explícita la mecánica de ritmo.
- La habilidad Mordedor se asigna a la tecla E en ambos niveles.
- La corriente del Nivel 2 se precisa como una combinación de componente oscilatoria sinusoidal más una componente constante de avance, en lugar de solo oscilatoria.

- La atracción de los remolinos se modela como fuerza inversamente proporcional al cuadrado de la distancia al centro, lo que genera el efecto de espiral de forma natural.
- Se abandono la idea de obstáculos en el nivel 2 por razones técnicas.

3. Sprites seleccionados

3.1 Nivel 1 — Piscina Olímpica de Cerulean City

- Totodile — spritesheet lateral con animación de brazadas (fuente: The Spriters Resource, Gen II sprites).
- Psyduck — spritesheet lateral con animación de nado (fuente: The Spriters Resource, Gen II sprites).
- Magikarp — sprite de salto con animación parabólica (fuente: The Spriters Resource, Gen II sprites).
- Fondo piscina — tileset de piscina con carriles, tribunas y cielo exterior (fuente: recursos libres de uso educativo, estilo pixel art Pokémon).

3.2 Nivel 2 — Río Safari, Zona Safari

- Totodile cenital — sprite visto desde arriba con animación de brazadas en cuatro direcciones (fuente: The Spriters Resource, Gen II overhead sprites).
- Remolino — sprite animado circular con efecto de espiral (fuente: recurso libre pixel art).

4. Diagrama de clases — capa lógica

El diagrama de clases de la capa lógica se adjunta como imagen en el directorio docs/ del repositorio (archivo diagrama_clases_logica_v1.png). A continuación se describe la arquitectura de forma resumida.

La jerarquía de herencia propia del proyecto parte de la clase abstracta Personaje, que define los atributos comunes de posición, velocidad y estado activo, así como el método virtual puro actualizar(dt). De esta clase heredan Jugador y AgenteRival, cada una con su comportamiento específico.

La clase Jugador extiende Personaje incorporando la stamina, los timers de las habilidades Mordedor e Hiperactivo, y el coeficiente de resistencia k que parametriza la física de decaimiento exponencial. Se decidió implementar esta física directamente dentro de la clase Jugador porque forma parte de su comportamiento y no era necesario crear una clase adicional para manejarla.

La clase AgenteRival extiende Personaje e implementa el ciclo del agente inteligente mediante cuatro métodos: percibir(), razonar(), actuar() y aprender(). El historial de tiempos del jugador se almacena en un `std::vector<float>` que persiste durante la sesión.

La clase `Obstaculo` representa cualquier elemento dinámico del nivel (Magikarps). Su tipo se determina con un entero y su física se ejecuta en el método `actualizar()` mediante un switch interno. Los parámetros de cada física se almacenan en un arreglo `float*` de tamaño fijo gestionado con memoria dinámica, por esta razón se utiliza memoria dinámica para almacenar dichos parámetros.

La clase `Nivel` contiene y maneja en composición al `Jugador`, al `AgenteRival` y a un `std::vector<Obstaculo*>` de obstáculos activos. Es responsable de actualizar todos los elementos, verificar colisiones y determinar cuándo el nivel ha terminado. El destructor de `Nivel` libera toda la memoria dinámica de los obstáculos.

La clase `Juego` orquesta los dos niveles en composición, maneja el estado global del juego (menú, jugando, fin) y sirve de interfaz entre la capa lógica y la capa GUI.

4.1 Relaciones entre clases

Desde	Hacia	Tipo	Multiplicidad
Jugador	Personaje	Herencia	— (herencia, sin multiplicidad)
AgenteRival	Personaje	Herencia	— (herencia, sin multiplicidad)
Juego	Nivel	Composición	1 Juego — 1 Nivel (en un momento dado)
Nivel	Jugador	Composición	1 Nivel — 1 Jugador
Nivel	AgenteRival	Composición	1 Nivel — 1 AgenteRival
Nivel	Obstaculo	Composición	1 Nivel — 0..* Obstaculo

4.2 Contenedores STL utilizados

Contenedor	Ubicación	Justificación
<code>std::vector<Obstaculo*></code>	Nivel	Colección de tamaño variable de obstáculos activos. Se eligió este contenedor porque permite agregar y eliminar obstáculos fácilmente durante la partida.
<code>std::vector<float></code>	AgenteRival	Historial de tiempos del jugador. Crece dinámicamente por sesión y se recorre secuencialmente para calcular el promedio.

5. Arquitectura del agente inteligente — Psyduck

El agente `Psyduck` implementa un ciclo de cuatro componentes tal como lo exige el documento guía del proyecto. A continuación se describe cada componente en detalle.

5.1 Percepción

Al finalizar cada carrera del Nivel 1, el sistema llama al método `percibir(tiempoJugador)` del agente pasándole el tiempo total que tardó el jugador en completar la carrera. El agente únicamente tiene en cuenta el tiempo que tarda el jugador en terminar la carrera. No percibe la posición instantánea del jugador durante la carrera, solo el resultado final.

5.2 Razonamiento

El método `razonar()` toma el historial de tiempos acumulado y calcula el promedio de las últimas N carreras (donde N es configurable, por defecto 3). Luego compara el tiempo más reciente con ese promedio: si el jugador mejoró (tiempo menor al promedio), el agente determina que debe volverse más agresivo. Si el jugador empeoró (tiempo mayor al promedio), el agente se relaja ligeramente. Si la diferencia es menor a un umbral de tolerancia, el agente mantiene su velocidad actual.

5.3 Acción

El método `actuar()` ajusta el atributo `velocidadBase` del agente según la decisión tomada en `razonar()`. El ajuste es proporcional a la diferencia entre el tiempo reciente y el promedio histórico, acotado por valores mínimo y máximo para que el agente no se vuelva imposible de vencer ni trivialmente lento. Este ajuste se aplica al inicio de la siguiente carrera.

5.4 Aprendizaje

El método `aprender()` agrega el tiempo más reciente al `std::vector<float>` `historialTiempos`. Este vector persiste durante toda la sesión de juego, de modo que el agente acumula información de todas las carreras jugadas. A medida que el historial crece, el promedio se vuelve más representativo del nivel real del jugador, de esta manera el agente tiene más información para ajustar su comportamiento en carreras futuras.

Inicialmente se consideró que Psyduck analizara la posición del jugador durante la carrera, pero se descartó porque aumentaba la complejidad sin aportar demasiado al comportamiento final.

Al principio se pensó en usar las flechas del teclado, pero se eligieron las teclas WASD para mantener los controles similares entre ambos niveles.

5.5 Flujo completo del ciclo del agente

- Jugador termina una carrera → sistema llama a `percibir(tiempo)`.
- `percibir()` almacena el tiempo temporalmente y llama a `razonar()`.
- `razonar()` calcula el promedio del historial y determina la acción.
- `actuar()` ajusta `velocidadBase` para la próxima carrera.
- `aprender()` agrega el tiempo al historial para futuras decisiones.
- Al inicio de la siguiente carrera, Psyduck nada con la nueva `velocidadBase`.

Repositorio: [https://github.com/\[usuario\]/pokemon-swim-championship](https://github.com/[usuario]/pokemon-swim-championship)